30 mai 2023

Microcontrôleurs

ECOLE POLYTECHNIQUE FEDERALE DE LAUSANNE

Projet de fin de semestre 2023

Microtechnique

Groupe 097 :

Adélaïde ROBERT : 346471

Louis BAIXAS : 343736

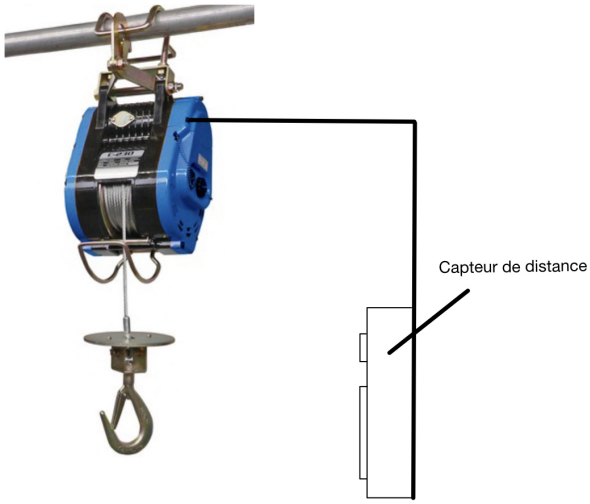
Encadrement :

Prof. A Schmid

Dans le cadre du projet de microcontrôleurs, nous avons réfléchi à l'élaboration d'une application faisant usage du MCU Atmega128L et de plusieurs périphériques du kit. Nous avons choisis de modéliser un "remonte-charge". Cette application fait intervenir le moteur Servo SG90, l'affichage LCD, la télécommande et le récepteur Infrarouge, le buzzer, une led et le détecteur de distance.

Le principe est de contrôler la montée/descente d'une charge (tel qu'une ancre de bateau par exemple) grâce à la télécommande (plusieurs modes sont possibles détaillés en section ..). La charge est remontée grâce au moteur Servo. Lorsque celle-ci est remontée au maximum, le capteur de distance la détecte, le mécanisme s'arrête automatiquement et un message s'affiche.

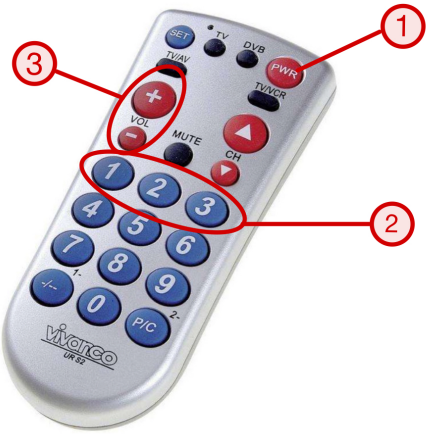
Nous avons choisi le moteur Servo SG90 car nous voulions pouvoir effectuer des rotations complètes avec de lourdes charges et n'avions pas besoin de contrôler précisément en valeur angulaire notre moteur.



1 Mode d'emploi

L'utilisateur utilise la télécommande pour contrôler la montée ou la descente de la charge grâce aux boutons. Tous les boutons configurés réagissent sans délais et leur utilité est présentée ci-dessous :

1. le bouton ON/OFF : ce bouton permet d'allumer le monte charge. Lorsqu'on presse le bouton, le mécanisme s'allume, affiche "on" et attend une instruction dictée par les autres boutons. Une led clignote toutes les 0.25s pour informer l'utilisateur que le mécanisme est allumé. Lorsqu'on presse une nouvelle fois le bouton, le mécanisme s'éteint, affiche "off" et plus aucune instruction n'est prise en compte, le moteur reste à l'arrêt, la led est atteinte.
2. Les chiffres un, deux et trois : ces boutons permettent de faire faire une rotation de 90,180 ou 360° respectivement au moteur. Ainsi l'utilisateur peut faire remonter la charge de 3 distances fixées.
3. Les boutons +/- : ces boutons permettent de faire monter ou descendre la charge (moins rapidement que les chiffres donc permettent de contrôler plus précisément la montée/descente). Ces boutons sont destinés à être pressés en continue jusqu'à ce que la charge atteigne la position voulue donc nous les avons configurés pour qu'ils réagissent sans délais et qu'on puisse rester appuyer sur la touche.



Lorsque la charge est montée au maximum, le capteur de distance détecte un mouvement et arrête automatiquement le moteur. Pour la sécurité, tant que la charge est détectée, on ne peut pas la montée davantage (autrement dit les boutons +, 1, 2 et 3 sont bloqués). Le bouton - en revanche est toujours actif pour pouvoir faire redescendre la charge.
Il faut ré-appuyer sur le bouton ON/OFF pour pouvoir la faire montée à nouveau.

2 Description technique de l'application et du matériel

2.1 Utilisation des ports et interfaces d'acquisitions

- Le module M4 avec le capteur de distance est connecté à la patte 3 du PORTF. En effet, le capteur de distance utilise une ligne analogique pour représentée la distance mesurée. La ligne analogique est connectée à une entrée du convertisseur analogique-numérique (PORTF).
- Le module M2 avec le buzzer et le détecteur Infra-Rouge est connecté sur le PORTE.
- Nous utilisons un deuxième module M4 pour connecter le moteur Servo car il ne fonctionne pas avec le convertisseur analogique-numérique donc ne peut pas être connecté au PORTF. Ce deuxième module M4 est branché sur le PORTD.

Nous n'utilisons pas les boutons de la carte, seulement 1 LED qui clignote lorsque le mode est "on" donc nous avons configuré le PORTB en sortie et connecté aux leds.

2.2 Affichage et son

A chaque fois qu'un bouton est pressé, l'affichage change (sauf lorsqu'on est en mode "off", les boutons sont inactifs donc on affiche "off" tant que le bouton ON/OFF n'est pas pressé à nouveau). En mode "on", lorsqu'on appuie sur le + ou le - : l'affichage indique "calibrage +" ou "calibrage-". Lorsqu'on appuie sur un chiffre 1 2 ou 3 : l'affichage indique "quart de tour" ou "demi tour" ou "tour complet".

A chaque fois qu'un bouton est pressé, un très cours signal sonore retenti, sauf pour les boutons + et - car ces boutons sont destinés à être pressé plusieurs fois d'affiler (sans relâcher par exemple) et un signal sonore à chaque fois serait pénible pour l'utilisateur.

Lorsque la charge est arrivée, l'affichage indique "CHARGE ARRIVEE" et si elle redescend,l'affichage reprend son comportement habituel (affichage correspondant au dernier bouton pressé).

2.3 Interuptions

Nous utilisons une interruption causée par un overflow du timer0 qui fait à la fois clignoter la led en mode ON et qui regarde si la charge est arrivé.

La source d'entrée du timer0 est le quarz horloger à 32'768Hz donc nous avons configurer le bit 3 du registre ASSR (AS0) à 1. Nous avons choisit un prescaler de 32 pour avoir un overflow toutes les 0,25s donc nous avons configurer les 3 premier bit du TCCR0 à 011.

L'interruption est suffisamment courte pour ne pas perturber la responsivité de notre

mécanisme. L'interruption qui vérifie si la charge est arrivée doit être très fréquente pour que la détection de la charge soit casi-instantanée. En effet, dans notre application on ne peut pas se permettre que la charge monte plus que le maximum donc la détection de celle-ci doit être très rapide. Nous avons dû optimiser au maximum les instructions de cette interruptions pour qu'elle s'effectue en un minimum de temps. En effet, un des critère du projet et que le programme réagisse en temps réel ce qui n'est plus le cas si l'interruption est trop longue puisque les commandes de l'utilisateur ne sont plus prises en compte pendant que le programme effectue l'interruption.

Notre application rempli ce critère puisque même lorsqu'on appuie sur un bouton de la télécommande en continue (le plus ou le moins par exemple), le comportement attendu à bien lieu et l'interruption est imperceptible.

Pour des raisons pratiques, on a choisi de mettre la led sur le même timer pour avoir une seule interruption donc en mode "on", la led clignote toutes les 0,5 ms.

Détail de l'interuption : L'overflow du timer0 est atteint toutes les 0,25s donc la routine d'interuption est appelée toutes les 0,25s. Elle commence par inverser un registre r29 puis inverse l'état de la led 1. Le fait d'inverser l'état du registre r29 permet d'appeler depuis le main une routine "captcheck" au lieu de l'appeler depuis la routine d'interruption qui doit rester très courte. Ainsi pendant que les instructions liées à l'interruption sont effectuées, le programme peut toujours recevoir d'autres instructions d'interruptions.

Nous utilisons notre capteur de distance comme un détecteur de mouvement : dès que la charge passe devant le capteur on veut que le moteur s'arrête. Mais le capteur détecte également les mouvement "d'arrière plan" qui ne doivent pas influencer le comportement du programme. Ainsi nous avons fixée une valeur "seuil" au-delà de laquelle le programme réagi. La routine "captcheck" lis la valeurs détectées par le capteur (voir section ... pour plus de détail sur le protocole) et la stocke dans les registre w et _w puis vérifie justement si la valeur "seuil" à été atteinte. Si c'est le cas, le registre r26 est mis à 0. Ainsi au prochain passage dans le main, le programme se comporte correctement : affichage et boutons bloqués sauf le - (descente).

2.4 Fonctionnement du programme

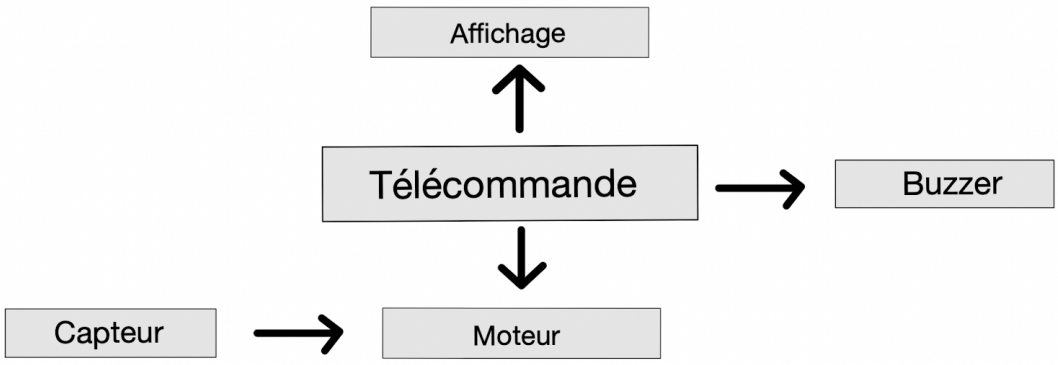
Le programme suit le principe des interruptions vectorisées. Dans les premières adresses de la mémoire programme se trouvent la table des vecteurs d'interruptions, suivies des routines d'interruption, suivies du programme principales.

Nous utilisons la pile uniquement pour stocker les valeurs d'adresses de retour des routines.

L'interruption "reset" nous sert à initialiser les ports d'entrées et de sorties, le Stack pointer, autoriser les interruptions d'overflow du timer 0 et définir deux constantes (fréquence de transmission de la télécommande et distance maximum de montée de charge).
Une fois rentré dans le programmes principale rien ne se passe tant qu'on a pas détécté de "start bit" sur la ligne IR (7) du que portE (macro WP1).
Ensuite, si une touche est pressée sur la télécommande, le signal est traité (voir section... pour les détails du protocole) et une valeur correspondante est stocké en mémoire donnée, dans les registres b0 et b1.
La macro "touche_compar" est appelé avec les differentes valeurs associer à chaque boutons, si un bouton correspond à la valeur contenu dans b0, b1 alors cette même macro redirige vers la bonne routine (contenant les instructions propres aux boutons correspondant). Par ex : si on presse +, après conversion la valeur stocké dans b1 et b2 est 111. On appelle la macro "touche_compar" plusieurs fois avec chaque valeur que pourrait contenir ces 2 registres. Lorsqu'on l'appelle avec l'argument 111, la comparaison est vérifiée et la routine "plus" est appelée.
En parallèle, l'interruption liée à la détection d'un mouvement appelle la routine "captcheck" toutes les 0,25s comme détaillé en section 2.3.
La mémoire programme contient aussi des adresses de données utilisées dans les routines d'affichages (.db).

3 Description de détail de l'accès au périphérique

3.1 Relations entre les périphériques



3.2 Description détaillée

Le programme utilise 5 périphériques et communique avec chacun d'entre eux grâce au port d'entrées-sorties.

Moteur Servo : Le PORTD, sur lequel est connecté le moteur Servo, est configuré en sortie (DDRD à 0xff). Plusieurs macro et routine ont été implémenté pour transmettre les bonnes données au moteur (CA3, _s360, servoreg_pulse et lpssp01). Nous communiquons sur le pin 4 du PORTD et la valeur de consigne au moteur est transmise sous forme de signal carré. Au repos, la ligne est à 0V et comprends toutes les 20ms des impulsions positives modulées en largeur de 1ms à 2ms. La modulation d'impulsion permet de définir le sens et la vitesse de rotation. Nous avons calibrer "par taton" pour obtenir les valeurs souhaitées (90°, 180°et 360°) qui ne sont pas très précises avec ce moteur servo mais ceci n'est pas un problème dans notre application.

Sur le PORTE est connecté le module M2 comprenant le détecteur Infra rouge et le buzzer. Il doit donc être configuré en entrée sur le pin 7 (télécommande) et en sortie sur le pin 2 (buzzer).

Buzzer : La communication avec le buzzer se fait simplement par l'envoie de signaux carrés sur cette ligne (la routine "buzzer" envoie un signal de 50 periodes de 2272 μs avec une symétrie des niveaux bas et haut).

Télécommande Infra-Rouge : Pour recevoir correctement le signal émis par la télécommande nous avons mesuré à l'oscilloscope la durée de transmission d'un bit. Nous avons mesurée 1810 μ s. La télécommande envoie des signaux selon le format de transmission RC5. Le code RC5 est composé de 14 bits encodés au format Manchester : un logic-1 est transmis si le signal passe du niveau logique bat au niveau haut au milieu de la période et un logic-0 est transmis si le signal passe du niveau haut au niveau bas au milieu de la période. Une transmission commence par deux bits de start, les cinqs bits suivant sont assignés au système adressé et les six derniers bits servent à transmettre des commandes. Ces 14 bits sont stockés dans les registres b0 et b1 par l'intermédiaire du Carry. Nous échantillonnons le premier signal au 3/4 de la période (3*1810/4 us) et les échantillonnages suivant se font toute les 1810us. Cette manoeuvre est la première étape du programme principale (main programme). Les délais sont respectée grâce a l'utilisation de macro WAIT.

Affichage LCD : Pour la communication avec l'affichage LCD, nous avons tirer partie du fichier *lcd.asm* qui contient une librairie de sous-routines permettant d'initialiser et de contrôler les

fonctions de base de l'affichage LCD. En particulier nous utilisons les routines LCD_home et LCD_init. Pour l'affichage des caractères nous avons utilisé la librairie *printf.asm* ce qui rend l'affichage très simple et compact : seulement une ligne de code invoquant la macro PRINTF ayant comme argument la destination donc LCD dans notre cas. Puis la ligne de code suivante est une directive à l'assembleur qui place en mémoire-programme les informations nécessaire à réaliser l'affichage, comprenant, le texte entre guillemets, le format dans un code spécifié et la variable (registre).

Capteur de distance : La communication avec le capteur de distance et le MCU se fait grâce à une conversion de signal analogique numérique. Les techniques de conversion requièrent un échantillonage du signal analogique à fréquence fixe. Le convertisseur analogique-numérique de l'ATmega128L utilise la méthode d'approximation successive. Cette technique détermine un nouveau bit à chaque cycle d'horloge de l'ADC, du MSB au LSB. Une conversion est déclanchée lorsque le bit ADSC du registre ADCSR est mis à 1 (donc dans notre cas lorsque la routine "captcheck" est appelé soit toute les 0.25s). Le résultat de la conversion est stockée dans les registres ADCH et ADCL (2 MSB et 8 LSB soit une résolution de 10 bits).

Nous avons configuer les registres ADPSx à 6 (un prescaler de 64) donc la conversion dure 208 μs. A la fin de la conversion, le bit ADSC est automatiquement remis à 0.

4 Annexe

4.1 Macros et sous-routines

Pour optimiser notre code, aussi bien en terme de lisibilité, que de rapidité et de stockage en mémoire, nous avons écris les macros et sous-routines adéquates. Leurs utilités sont présentés ci-dessous :

Routines/Macros	Modifiés/Arguments	Description
Routines d'interuptions		
Overflow0	modifiés : r29, led 1	Inverse r29 et l'état de LED1 (voir détail section 2.3)
Routines contrôle de la télécommande		
TOUCHE_COMPAR	arguments : nombre, routine modifié : flag Z	Compare le nombre avec b0 et execute la routine argument si la comparaison est vérifiée
TOUCHE_PM	arguments : temps en μs, routine	Envoie la bonne largeur d'impulsion sur la ligne qui contrôle le moteur et exécute la routine argument(affichage)
TOUCHE_CHIFFRE	arguments : nombre	Exécute la routine buzzer puis appelle les routines de contrôle du moteur
power	modifié : r26	Execute la routine buzzer, inverse r26 pour changer de mode (ON/OFF) appelle la routine d'affichage
buzzer	modifié : r30	Evoie un signal carré sur la ligne du buzzer pour emetre un son
Routines contrôle du capteur de distance		
captcheck	modifiés : r29, w, _w, flag Z	place la valeur convertie par le convertisseur analogique dans w et _w et la compare avec une valeur seuil : si cette valeur est atteinte (charge detectée), la routine stop est executée
stop	modifiés : r26	charge r26 à 0 pour que les boutons soient désactivés, affiche "CHARGE ARRIVE"
power_affiche	modifié : _sreg	appelle on ou off en fonction de la valeur de r26
on et off	modifié : bit I de SREG, r28	affiche "on" ou "off", charge I de SREG à 1 ou 0, charge r28 à 0 ou 1 respectivement si on est en mode on ou off.

Pour ne pas surcharger ce tableau, nous avons choisit de ne pas présenter les routines d'affichages : printplus, printmoins, un, deux, trois. Ces routines ne font qu'afficher respectivement "MONTEE", "DESCENTE", "quart de tour", "un demi tour" ou "tour complet". Les routines un, deux, et trois appellent également la macro TOUCHE_CHIFFRE avec les arguements correspondnat la touche 1, 2 ou 3 de la télécommande.

Pour le contrôle du moteur, nous avons également choisit de les présenter à part : en effet, leur fonctionnement est assez particulier et difficile à résumer. Tout d'abord la macro CA3 est appelé, lorsqu'une touche est pressée. Cette macro prend comme argument, une routine (_s360), un nombre (1750) et un nombre (correspondant à la touche pressée). Ce nombre est chargé dans b0, et les registres a0 et a1 sont chargé à 1750 puis la macro appelle _s360. Cette routine decremente b0 et appelle servoreg_pulse tant que b0 n'est pas nul. C'est la routine servoreg_pulse qui "écrit" la ligne de contrôle du moteur suivant le paramètre b0. Le détail du fonctionnement de cette ligne est expliqué en section 3.

4.2 Programme

4.3 Module principal

buzzer.asm | capteurdistance.asm | telecommande.asm | motor.asm | projet.asm*

```
; created: 17/05/2023 09:48:04
; Author : louis
; module main

.include "definitions.asm"      ; register/constant definitions
.include "macros.asm"           ; macro definitions


; === interrupt table ===

.org 0
    rjmp reset
.org OVF0addr
    rjmp overflow0


; === interrupt service routines===
overflow0:
    com r29                     ; capteur de distance activée lorsque r29 = 0xff
    INVP PORTB, 1               ; clignotement de la led 1
    reti

; === Initialisation===
reset:
    LDSP RAMEND                 ; Initialise le stack pointeur
    clr r29                     ; eteint le capteur
    clr R26                     ; Initialise en mode off
    OUTI DDRD, 0xff             ; PORTD en sortie, moteur
    sbi DDRE,SPEAKER            ; BUZZER en sortie,
    OUTI DDRB,0xff              ; PORTB en sortie,
    OUTI PORTB,0xff             ; eteit les led
    OUTI TIMSK,(1<<TOIE0)       ; Overflow 0 autorisé
    OUTI ASSR, (1<<AS0)         ; selection de l'orloge externe
    OUTI TCCR0,3                ; prescaleur = 32, overflow toute les 0.25secondes

    OUTI ADCSR, (1<<ADEN) + 6   ; Initialisation du convertisseur analogique + PS = CK/64
    OUTI ADMUX,3                ;

    .equ T1=1810                ; T1 = frequence télécommande
    .equ konst=0x0100           ; palier capteur de distance

    rcall LCD_init              ; initialise l'ecran lcd
    rjmp main                   ; saut vers le main


.include "motor.asm"
.include "lcd.asm"
.include "printf.asm"
.include "telecommande.asm"
.include "capteurdistance.asm"
.include "buzzer.asm"

;=== programme principale===
main :
    sbrs r29,0                  ; verification de l'overflow timer 0
    rcall captcheck
    CLR2 b1, b0                 ; clear 2-byte register
    ldi b2,14                   ; load bit counter
    WP1 PINE, IR                ; wait if PIN= 1

    WAIT_US (3*T1/4)            ; wait tree quarter period

loop :
    P2C PINE, IR                ; move pin to carry (P2C)
    ROL2 b1, b0                 ; role carry into 2-byte register
                                ; >reg
    WAIT_US (T1-4)              ; wait a bite periode
                                ; >(- compensation)
    DJNZ b2, loop               ; decrement and jump if
                                ; >Not zero

    TOUCHE_COMPAR 115, power
    cpi r28,1                   ; vérifie le flag qui indique si l'utilisateur a presser on ou off
    breq mode_check
    TOUCHE_COMPAR 110, moins
mode_check:
    cpi r26, 0x00               ; vérifie si le flag qui indique si le programme est en on ou off
    breq main

on1:

    TOUCHE_COMPAR 110, moins
    TOUCHE_COMPAR 111, plus
    TOUCHE_COMPAR 126, un
    TOUCHE_COMPAR 125, deux
    TOUCHE_COMPAR 124, trois
    rjmp main
```

4.4 Module télécommande

buzzer.asm | capteurdistance.asm | telecommande.asm | motor.asm* | projet.asm*

```asm
 * telecommande.asm
 *
 *  Created: 29/05/2023 16:11:30
 *   Author: louis
 */
 .macro TOUCHE_COMPAR           ; saute vers arg1 si arg0 = b0

    cpi b0, @0
    in  r1,SREG
    sbrc r1,1
    rjmp @1
    clr r1
.endmacro

.macro TOUCHE_PM                ; envoie un signal carré sur le Pin SERVO1 du PORTD
                                ; de largeur d'impulsion arg0 et saute vers arg1
    P1 PORTD, SERVO1
    WAIT_US @0
    P0 PORTD, SERVO1
    rjmp @1
.endmacro

.macro TOUCHE_CHIFFRE           ; saute vers buzzer,actionne le moteur avec arg1 comme angle

    rcall buzzer                ; saute vers buzzer
    ldi b0, @0
    CA3 _s360, 1750,@1
    rjmp main                   ; saute vers main
.endmacro

power:
    rcall buzzer                ; saute vers buzzer
    com r26                     ; inverse r26 pour passer en mode on ou off
    WAIT_MS 500                 ; attend 500 ms
    rcall power_affiche         ; appelle power affiche


power_affiche:
    cpi r26,0                   ; mode OF si r26 = 0, mode ON si r26 different
    in _sreg,SREG
    sbrc _sreg, 1
    rjmp off

on:
    sei                          ; active les interuptions
    clr r28                      ; flag du mode off dans le main desactivé
    rcall LCD_home               ; affichage de on

    PRINTF LCD
    .db "on              ", 0
    rjmp on1

off:
    ldi r28,0b00000001           ; flag du mode off dans le main activé
    cli                          ; désactive les intéruptions
    OUTI PORTB,0xff              ; éteint les leds
    rcall LCD_home               ; affichage de of

    PRINTF LCD

    .db "off             ", 0
    rjmp main

printplus:                       ; affichage de "MONTE           "
    rcall LCD_home

    PRINTF LCD

    .db "MONTEE         ", 0
    rjmp main

printmoins:                      ; affichage de "DESCENTE           "
    rcall LCD_home

    PRINTF LCD

    .db "DESCENTE        ", 0
    rjmp main


; appelle une souroutine qui fais bouger le moteur et une souroutine d'affichage
plus:
touche_pm 2000, printplus

; appelle une souroutine qui fais bouger le moteur et une souroutine d'affichage
moins:
```

```
; appelle une souroutine qui fais bouger le moteur et une souroutine d'affichage
moins:
TOUCHE_PM 1000, printmoins

; appelle une souroutine qui fais bouger le moteur et "affiche quart de tour"
un:
    rcall LCD_home

    PRINTF LCD

    .db "quart de tour   ", 0
    TOUCHE_CHIFFRE 1, 0x07

; appelle une souroutine qui fais bouger le moteur et affiche "un demi tour"
deux:
    rcall LCD_home

    PRINTF LCD

    .db "un demi tour    ", 0
    TOUCHE_CHIFFRE 2, 0x10

; appelle une souroutine qui fais bouger le moteur et affiche "tour complet"
trois:
    rcall LCD_home

    PRINTF LCD

    .db "tour complet    ", 0
    TOUCHE_CHIFFRE 3, 0x21
```

4.5 Module Buzzer

buzzer.asm | capteurdistance.asm | telecommande.asm | motor.asm* | projet.asm*

```
/*
 * buzzer.asm
 *
 *  Created: 29/05/2023 16:45:41
 *   Author: louis
 */
 buzzer:                            ; emet un son
ldi r30, 50                     ; initialise le conteur a 30
label:
    sbi PORTE, SPEAKER          ; emission du signal carré
    WAIT_US 1136                ; largeur d'impulsion
    cbi PORTE, SPEAKER          ; fin du signal
    WAIT_US 1136
    dec r30                     ; dercrémentation du conteur
    brne label                  ; branche si le compteur est a 0
    ret
```

4.6 Module Moteur

buzzer.asm | capteurdistance.asm | telecommande.asm | motor.asm | projet.asm*

```
/*
 * motor.asm
 *
 *  Created: 29/05/2023 16:07:30
 *   Author: louis
 */
 ; _s360, in a1:a0, b0 out void, mod a2,w
 ; purpose execute arbitrary rotations
 _s360 :
    rcall servoreg_pulse
    dec b0
    brne _s360
    ret
; servoreg_pulse, in a1,a0, out servo port, mod a3,a2
; purpose generates pulse of length a1,a0
servoreg_pulse :
    WAIT_US 20000
    MOV2 a3, a2, a1, a0
    P1 PORTD, SERVO1

lpssp01:
    DEC2 a3, a2
    brne lpssp01
    P0 PORTD, SERVO1
    ret

.macro CA3                      ; appel une sous-routine avec 3 arguments
                                ; arguments dans a0:a1,b0
    ldi a0, low(@1)             ;vitesse et sens de rotation
    ldi a1, high(@1)            ;vitesse et sens de rotation
    ldi b0, @2                  ;angle
    rcall @0
.endmacro
```

4.7 Module Capteur

buzzer.asm | capteurdistance.asm | telecommande.asm | motor.asm* | projet.asm*

```
/*
 * capteurdistance.asm
 *
 *  Created: 29/05/2023 16:17:23
 *   Author: louis
 */
 captcheck :                        ; controle de la distance
    clr r29                     ; r29 a 0, le programe ne sautera plus dans captcheck au prochain appel du ma
    sbi ADCSR, ADSC             ; déclenche une conversion sur le capteur de distance
    in w, ADCL                  ; place le signal échantillonée dans w et _w
    in _w, ADCH
    cpi _w,0
    brne stop                   ; si le signale est superieur ou egale a 0x0100, saut vers stop,mode off acti
    cbi ADCSR, ADSC             ; désactive la conversion
    ret

stop:
    ldi r26, 0x00               ; mode off au prochain passge dans le main
    rcall LCD_home              ; affiche "CHARGE ARRIVE"
    PRINTF LCD
    .db "CHARGE ARRIVE   ", 0
    ret
```